

QUESTION 9(a)

Question:

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main()
{
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX];
    char path[PATH_MAX];

    /* Get initial working directory */
    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(previous_dir, current_dir);

    printf("==== Directory Navigation =====\n");
    printf("Type 'pwd' to show current directory\n");
    printf("Type 'cd <path>' to change directory\n");
    printf("Type 'prev' to return to previous directory\n");
    printf("Type 'exit' to quit\n");

    while (1)
    {
        printf("\n$ ");

        char command[PATH_MAX];

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';
```

```
        if (strncmp(command, "cd ", 3) == 0)
        {
            strcpy(path, command + 3);

            /* Validate path before changing */
            if (access(path, F_OK) != 0)
            {
                printf("Error: Path does not exist.\n");
                continue;
            }

            if (access(path, X_OK) != 0)
            {
                printf("Error: Permission denied.\n");
                continue;
            }

            /* Store current directory */
            if (getcwd(current_dir, sizeof(current_dir)) == NULL)
            {
                perror("getcwd");
                continue;
            }

            /* Change directory */
            if (chdir(path) == 0)
            {
                strcpy(previous_dir, current_dir);

                if (getcwd(current_dir, sizeof(current_dir)) != NULL)
                {
                    printf("Directory changed successfully.\n");
                    printf("Current Directory: %s\n",
                        current_dir);
                }
            }
            else
```

Output

```
sri_dhruthi@Dhruthi:~$ nano skill9a.c
sri_dhruthi@Dhruthi:~$ gcc skill9a.c -o skill9a
sri_dhruthi@Dhruthi:~$ ./skill9a
==== Directory Navigation ====
Type 'pwd' to show current directory
Type 'cd <path>' to change directory
Type 'prev' to return to previous directory
Type 'exit' to quit

$ pwd
Current Directory: /home/sri_dhruthi

$ cd /tmp
Directory changed successfully.
Current Directory: /tmp
```

QUESTION 9(b)

Question:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX 200

void cmd_pwd()
{
    char path[MAX];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("Current Directory: %s\n", path);
    else
        perror("pwd");
}

void cmd_cd(char *path)
{
    if (path == NULL)
    {
        printf("Usage: cd <directory>\n");
        return;
    }

    if (chdir(path) != 0)
        perror("cd");
    else
        printf("Directory changed successfully.\n");
}

void cmd_echo(char *text)
{
    if (text != NULL)
        printf("%s\n", text);
}
```

```

struct Builtin
{
    char *name;
    void (*function)();
};

int main()
{
    char input[MAX];
    char *command;
    char *argument;

    /* Dispatch table */
    struct Builtin builtins[] =
    {
        {"pwd", cmd_pwd},
        {"help", cmd_help}
    };

    int builtin_count = 2;

    printf("==== Built-in Command Shell ==== \n");
    printf("Type 'help' for available commands. \n");

    while (1)
    {
        printf("\n$ ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        /* Ignore empty input */
        if (strlen(input) == 0)
            continue;

```

```

        /* Ignore empty input */
        if (strlen(input) == 0)
            continue;

        command = strtok(input, " ");
        argument = strtok(NULL, "");

        /* Exit command */
        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell... \n");
            break;
        }

        /* cd */
        if (strcmp(command, "cd") == 0)
        {
            cmd_cd(argument);
            continue;
        }

        /* echo */
        if (strcmp(command, "echo") == 0)
        {
            cmd_echo(argument);
            continue;
        }

        /* Search dispatch table */
        int found = 0;

        for (int i = 0; i < builtin_count; i++)
        {
            if (strcmp(command, builtins[i].name) == 0)
            {
                builtins[i].function();
                found = 1;

```

Output

```
Program terminated.  
sri_dhruthi@Dhruthi:~$ nano skill9b.c  
sri_dhruthi@Dhruthi:~$ gcc skill9b.c -o skill9b  
sri_dhruthi@Dhruthi:~$ ./skill9b  
==== Built-in Command Shell ====  
Type 'help' for available commands.  
  
$ pwd  
Current Directory: /home/sri_dhruthi  
  
$ cd /tmp  
Directory changed successfully.  
  
$ echo Hello World  
Hello World  
  
$ help  
  
==== Built-in Commands ====  
cd <dir>      - Change directory  
pwd           - Show current directory  
echo <text>   - Display text  
help         - Show available commands  
exit         - Exit shell  
  
$ hello  
Invalid command: hello  
  
$ exit  
Exiting shell...
```